GENERATIVE AI FOR SOFTWARE ENGINEERING

CAPSTONE PROJECT

PROJECT CORPUS FORGE

MISSION

Design and implement a web-based system that transforms a heterogeneous corpus into actionable knowledge through:

- retrieval,
- prompting.

Your system must enable users to import, organize, explore, and interact with collections of documents using AI-powered workflows.

The goal of this project is **not** simply to build an application.

The goal is to demonstrate your ability to:

- learn unfamiliar technical concepts quickly,
- use AI-assisted development tools effectively,
- make sound engineering decisions under uncertainty,
- evaluate trade-offs,
- and deliver a working software system.

LEARNING OBJECTIVES

By completing this project, you should demonstrate your ability to:

- design and implement an AI-powered software system;
- apply Retrieval-Augmented Generation (RAG) techniques to different document types;
- engineer prompts to produce reliable and useful outputs;
- use AI-assisted development: Visual Studio Code Copilot, as learning, debugging, design, and implementation partners;
- explain a system architecture and justify engineering decisions;
- critically evaluate both your system and the AI tools used during development.

TEAM STRUCTURE AND TIMELINE

- Teams of **3 students**
- Deadlines:
  - GitHub Repos Submission: **May 22, 10:00 PM**
  - Presentation: **May 28** (see Presentation Guidelines below)

GITHUB REPOSITORY

- One repository per team.
- One member of the team:
  1. clones https://github.com/bcs-s2-2026/capstone-corpus-forge
  2. creates a new GitHub repository named 'capstone-corpus-forge'. The repository must be public to give easy access to the other members of the team.
  3. links the cloned capstone project repository (1) to the new repository in (2).
- The other members of the team clone the newly created GitHub repository (2).
- The team repository must be created and sent to me by the end of today's class.
- Each team member clones and pushes to the same repository.
- Apply good source control and software engineering practices:
  - meaningful commit messages. Use 'feat', 'fix', 'refactor', 'docs', etc. prefixes to indicate the type of change.
  - commit early and often.
  - you may use branches if you are comfortable with Git, but the main branch should always be in a runnable state.

PROJECT REQUIREMENTS

Your project is composed of **two layers:**

LAYER 1 — CORE PLATFORM (REQUIRED)

You must implement a **working web application**.

The application does not need polished visual design, but the user experience must be functional, coherent, and allow access to all implemented features.

Your platform must support the following core capabilities.

1. CORPUS INGESTION

Users must be able to import and process documents from at least **3 different file types**, including:

- plain text and/or markdown
- PDF
- source code files

Examples:

- `.txt`
- `.md`
- `.pdf`
- `.py`
- `.js`

The system must process these documents into a form suitable for retrieval and AI interaction.

2. CORPUS MANAGEMENT

Users must be able to:

- add documents
- remove documents
- browse available documents
- select which documents are active for AI interactions

3. RETRIEVAL-GROUNDED KNOWLEDGE EXPLORATION & ARTIFACT GENERATION

Users must be able to interact with the selected corpus through AI-powered, retrieval-grounded workflows.

The system must support the following corpus-grounded features:

- Chat-based exploration and question answering
- Interactive Flashcards and Quizzes
- For source code corpora:
  - Code Review Report
  - Architecture and Control Flow Report

Flashcards, quizzes, and reports must persist between application runs.

Users must be able to influence generation through prompts or parameters, as described in the next section.

4. PROMPT STEERING

Users must be able to influence AI behavior through customizable prompts or parameters.

Examples:

- audience level
- response format
- creativity level
- tone
- task instructions

5. PERSISTENCE

Documents, metadata, and generated artifacts must persist between application runs.

6. COST OBSERVABILITY

Your application must expose AI usage information.

At minimum:

- number of requests
- token usage

Additional metrics are encouraged.

LAYER 2 — ENGINEERING CHALLENGES (CHOOSE ANY 2)

Each team must select **two engineering challenges**.

You are encouraged to select challenges that push your technical comfort zone.

CHALLENGE A — RETRIEVAL EXPERIMENTATION

Implement and compare at least **two retrieval strategies**.

Examples:

- fixed-size, overlapping chunking
- semantic chunking
- file-level retrieval
- function-level retrieval
- hybrid keyword + vector search
- metadata filtering

You must evaluate:

- what worked
- what failed
- when each strategy is appropriate

CHALLENGE B — PROMPT ENGINEERING

Iteratively improve prompts for a specific task.

Examples:

- reducing hallucinations
- improving quiz quality
- improving code explanations
- improving citation quality

You must document prompt evolution.

CHALLENGE C — VISUALIZATION

Generate an interactive visualization derived from the corpus.

Examples:

- word cloud
- concept graph

Users must be able to interact with the visualization.

CHALLENGE D — RELIABILITY AND TESTING

Design tests for your system.

Examples:

- unit tests
- integration tests
- edge-case tests
- malformed input handling

You must intentionally test failure scenarios.

Examples:

- empty corpus
- duplicate files
- conflicting documents
- corrupted files

TECHNOLOGY CONSTRAINTS I

You may choose your own:

- programming language(s)
- frameworks
- database(s)
- frontend technologies
- backend technologies: teams are strongly encouraged to use Python unless they can justify an alternative stack.

You may use:

- Google GenAI APIs
- vector databases such as ChromaDB
- AI-assisted development: VS Code Copilot.
  - Do not use Claude or Codex or ChatGPT or any other AI code generation tool unless it has been discussed and approved by me. In that case, you must document how you used it in the AI Collaboration section of your report and include your prompt history in the deliverables.
- Your own custom agents (Make sure to document and mention them in your report)

TECHNOLOGY CONSTRAINTS II

You may use AI to:

- learn
- design
- debug
- implement
- test
- document

Make sure you break down your implementation into small, manageable tasks and use AI to assist with specific problems or questions that arise during development.

However, you may not use AI to:

- generate the entire system or large components in one step.
- generate the project's final REPORT.md (see Deliverables section below).

DELIVERABLES

1. WORKING APPLICATION

A runnable web application running locally on the user's machine.

2. ARTIFACTS AND DOCUMENTATION

- GitHub repo with source code, documentation, and instructions to run the application locally.
- Documents:
  - README.md with setup instructions
  - REPORT.md with technical report (see below)
  - Prompt history file(s):
    - JOURNAL.md
    - prompt_history.md
  - Architecture, Code Explorer, Flashcards and Quiz Custom Agents outputs in a folder called 'docs'.
  - Presentation slides (see Presentation Guidelines below)
    - Group Presentation named `group_presentation.pdf`
    - Individual Presentation named `individual_presentation_student_name.pdf`
    -

TECHNICAL REPORT (REPORT.md)

Your report must document:

THE TEAM MEMBERS

- Names, EPITA email addresses, and GitHub usernames of all team members.

INITIAL DESIGN

- initial architecture
- assumptions
- technical choices

ENGINEERING DECISIONS

For each major decision:

- what alternatives were considered?
- why was this solution chosen?

WHO DID WHAT?

- Document how the project was originally divided among each team member.
- Document how responsibilities possibly evolved over time.

AI COLLABORATION

Document how AI tools were used.

- What tools were used for what purposes?
- How did AI influence design and implementation decisions?
- How did AI impact your learning and development process?
- How did you evaluate AI-generated suggestions?
- How did you detect and handle AI errors or limitations?

FAILURES AND ITERATIONS

Document:

- what failed?
- what surprised you?
- what required redesign?

“WHEN AI FAILED OR WAS WRONG”

Document cases where AI-generated advice, code, or explanations were:

- incomplete
- misleading
- incorrect
- inefficient

Explain how you detected the issue and how you resolved it.

LESSONS LEARNED

Reflect on:

- technical growth
- workflow improvements
- Strengths and limitations of AI-assisted development

PRESENTATIONS

- Each team will give a **15-minute presentation** of their project, followed by a **5-minute Q&A** session for a total of 20 minutes per team.

- Breakdown of presentation time:
  - Demo: 3 minutes
  - Group Presentation + Individual Contributions: 12 minutes
    - Main Presentation: 3 minutes
    - Individual Presentations: 3 x 3 minutes = 9 minutes
    - Or you may choose to do a single group presentation where each member contributes to different sections, as long as the total time is 12 minutes.
  - Q&A: 5 minutes

- Notes: For those who anticipate not being able to attend the presentation on the scheduled date (May 28):
  - Please inform Scolarité and me as soon as possible.
  - Plan on attending remotely.
  - You will be expected to present your individual contributions during the presentation, so make sure to coordinate with your team and prepare your slides accordingly.

IMPORTANT

This project evaluates your **engineering judgment**, not just your ability to ship features. A "perfect" application that you cannot explain is worth less than a complex challenge accompanied by a rigorous post-mortem.

I prioritize:

- **AI-Enhanced Learning:** Using AI as a high-fidelity tutor to master new concepts and technologies, rather than a shortcut to bypass the learning process.
- **Evidence-Based Decisions:** Justifying your architecture and prompt choices through experimentation and the "paper trail" in your logs.
- **Honest Iteration:** I value the documentation of a maojr failure and the subsequent redesign more than a project that worked on the first try by playing it safe.

Bottom Line: Higher marks are awarded to teams that demonstrate a "smart" partnership with AI—leveraging it to accelerate their technical growth—and can critically analyze the trade-offs of their final system.